

A Resource-Bounded Hybrid Small Language Model

DASE7506 MP1 | Student ID: 3036707472 | GitHub: jiucaihehe
Protocol: 7506-mp1-wt2-v2

1. Objective and main result

This project trains small GPT models from random initialization on the supplied WikiText-2 training text. It combines neural prediction, a strictly within-window continuous cache, and compact interpolated Kneser-Ney statistics. Validation selects mixture settings and the final checkpoint. The final predictor and every reported ablation are frozen before any test score is inspected.

The submitted predictor obtains **1.539148 full-test FP32 BPB**, compared with 2.101265 for the initial classroom baseline (26.75% lower). This total gain includes architecture, training and prediction changes. Matched-weight ablations isolate the contribution of each prediction component; the total improvement is not attributed to one mechanism.

Final result	Measured value
Validation BPB	1.522428
Full-test BPB	1.539148
CPU scoring / baseline	2.454x (limit 5x)
Peak evaluation RAM	2.197 GiB (limit 4 GiB)
Uncompressed inference assets	60.219 MiB (limit 64 MiB)

The submitted code includes the unchanged course data, tokenizer and scorer, a directly evaluable checkpoint, training and search logs, source/checkpoint hashes, and exact reproduction commands. The method uses no external training text, pretrained weights, cached validation/test answers, cross-window state or evaluation network access.

The implementation adapts established ideas: continuous caches [1] and Kneser-Ney smoothing [2,3]. The contribution is their constrained implementation, combination, resource accounting and controlled evaluation for this assignment, rather than a claim to invent these methods.

2. Predictor and causal boundaries

The selected GPT has width 256, depth 4, 4 attention heads, learned position embeddings, tied input/output embeddings and vocabulary 2,048. Context length remains 256. Training dropout is 0.1; dropout is disabled during scoring. The neural distribution is $\text{softmax}(W h[t] / T)$, with $T=1.1$.

Within-window cache. A cache key $h[s]$ is paired with observed token $x[s+1]$. At query position t , only keys with $s < t$ are eligible, so each cached value is already visible in the input prefix. Similarity weights are a softmax of cosine similarities with optional recency decay:

```
a[t,s] = softmax(s < t){theta * cos(h[t],h[s]) - decay*(t-s)}
p_C(w|t) = sum(s < t) a[t,s] * 1{x[s+1] = w}
p_NC = (1-lambda) * p_GPT + lambda * p_C
```

The chosen lambda is 0.05, theta is 16, and decay is 0. Position zero uses only GPT because the cache is empty. Each window and batch example constructs its own cache on every call. A strict triangular mask excludes future information. There is no persistent evaluation-prefix state.

Train-derived short-sequence statistics. A 5-gram model is fitted only to the training token sequence. The highest order uses occurrence counts; lower orders use counts of distinct left continuations. The unigram continuation distribution receives a 0.01 pseudocount for finite probabilities. One absolute discount D is used in the recursive interpolated distribution:

```
p_KN(w|h) = max(c(h,w)-D,0)/c(h) + b(h)*p_KN(w|suffix(h))
```

All order-2, order-3 and order-4 events are retained. Order-5 events need raw count at least two. Backoff mass $b(h)$ is one minus the total retained discounted probability, so removed events transfer mass to the lower-order distribution and normalization is preserved. Sorted exact integer keys represent contexts without hash collisions. Keys, pointers, counts and token IDs use bounded integer arrays.

Hybrid mixture. $p = (1-\alpha) p_{\text{NC}} + \alpha p_{\text{KN}}$, using $D=0.5$ and maximum order 5. The constant alpha is 0.2. When enabled, alpha depends on the longest matched stored context; selected weights for orders 2-5 are [0.2, 0.2, 0.2, 0.3]. Matching uses only the current prefix. These four finite-grid settings are selected on validation.

3. Training, selection and disclosed costs

All gradient runs use AdamW with weight decay 0.1, 100-step warmup, cosine decay to 10% of peak learning rate and gradient clipping at 1.0. Each update processes 32 sequences of 256 targets. Seed 17 is used throughout. The original baseline and 192-wide experiments use CPU FP32. The regularized 256-wide experiment uses the local Apple M5 GPU through MPS, with validation performed by the unmodified CPU FP32 scorer.

The 192-wide continuation restarts AdamW at peak learning rate 0.0002 and advances the deterministic sampler past ancestral targets. Other full runs use peak learning rate 0.001. Validation selects saved checkpoints. The 20-step MPS smoke run checks compatibility and is excluded from model selection but included in search cost.

Run	Device	New targets	Cumulative targets	Train seconds
baseline-s17	cpu	9,830,400	9,830,400	191.2
continued-s17	cpu	19,660,800	49,152,000	1072.8
mps-smoke	mps	163,840	163,840	14.3
regularized256-s17	mps	49,152,000	49,152,000	1676.1
wide-s17	cpu	29,491,200	29,491,200	1236.8

Total new gradient targets: **108,298,240**. Logged optimization time: 4191.2 s, excluding measured validation scoring. The final checkpoint has 49,152,000 cumulative gradient targets. Reusing checkpoints never resets ancestry. Concurrent CPU/GPU work can affect these elapsed training times; final resource measurements are isolated.

The statistical component scans 3,613,342 training targets. Its retained build took 13.07 s. A preliminary, more heavily pruned build took 13.27 s and was replaced using asset-size information before either statistical model was scored. The final stored asset is 45.890 MiB uncompressed.

All validation searches together took 228.09 s. Cache search tests 153 deduplicated combinations per backbone: T in {0.9,1,1.1}, theta in {0,4,8,16,32}, decay in {0,0.01}, lambda in {0,0.05,0.1,0.2,0.3,0.4}. Statistical mixture search tests 55 combinations of discount {0.5,0.75,0.9}, order {3,4,5} and weight {0,0.1,...,0.6}. Optional gating independently selects one of nine weights {0,0.1,...,0.8} in each of four buckets. Search JSON retains every aggregate result; prediction answers are not stored as inference assets.

The final method is selected solely by full validation BPB among the recorded candidates. Validation has 376,599 scored targets and 1,148,007 bytes; test has 428,405 targets and 1,292,013 bytes. BPB is total natural-log loss divided by $\ln(2)$ and the split raw-byte count. Every target except the first is scored exactly once, including the final short window.

4. Matched-budget controls and ablations

Frozen predictor	Validation BPB	Test BPB
Initial classroom baseline	2.071087	2.101265
Same initial baseline + cache	1.999506	2.012973
Selected GPT, T=1	1.595861	1.614482
Selected GPT, same T	1.588980	1.606800
Remove statistical mixture	1.566724	1.578519
Remove continuous cache	1.542126	1.564512
Uniform cache, same mixture	1.547729	1.569081
Constant statistical weight	1.523537	1.540046
Final selected hybrid	1.522428	1.539148

The initial baseline/cache pair has exactly the same neural weights and 9,830,400 gradient targets. For final-model ablations, all fitted components and their recorded fitting costs are shared: only the designated inference setting changes. Removing the cache sets lambda to zero. Removing the statistical mixture sets alpha to zero. The uniform-cache control sets theta to zero. Constant-mixture control removes the four per-order weights, retaining the previously selected constant alpha. Ablations are not re-tuned on test.

Removing the statistical component changes test BPB from 1.539148 to 1.578519; removing the continuous cache changes it to 1.564512. The pure selected GPT scores 1.606800. These matched comparisons identify the value of the components for the submitted checkpoint, without attributing gains to additional gradient training.

Statistical fitting is disclosed separately from gradient targets because the counting procedure scans the training text. The corresponding inference ablation disables an already-fitted component; it does not erase that shared preprocessing cost. A wider/longer-trained predictor compared with the original baseline is not a clean test of width, dropout or training duration individually.

Validation diagnostics compare the final model against its no-cache ablation while retaining the same statistical component and interpolation settings. The cache helps when useful continuations recur, but redistributing probability toward cached tokens can hurt new continuations. This is why a small cache weight can outperform a large one.

Validation target group	Targets	Nats saved by cache / target
Already in eligible cache	147,388	+0.16706
Absent from eligible cache	229,211	-0.03904

5. Resources, correctness and reproduction

Resource	Measured	Limit
Median baseline scoring	4.232 s	Reference on same host
Median final scoring	10.384 s	Ratio 2.454x <= 5x
Peak whole-process RAM	2.197 GiB	4 GiB
Uncompressed inference assets	60.219 MiB	64 MiB

Timing uses three fresh processes each for baseline and final, alternating on the same Apple M5 CPU with four PyTorch threads, FP32, and no concurrent training. The scorer seconds exclude imports/loading. Peak RAM is whole-process `ru_maxrss`, including loading, converted using macOS byte units. Compressed NPZ contents are counted at their expanded member sizes. The final checkpoint and all predictor modules are counted; fixed benchmark files, installed libraries and optional experiment evidence are excluded. Results on the instructor's Xeon may differ.

The original contract tests verify causality, normalized probabilities, independent examples, state reset, gradients and exact target coverage. Additional tests compare every prefix with full-window predictions, check strict cache masking, verify normalized pruned statistical distributions, test independent hybrid calls and confirm that dropout is inactive at evaluation. Original fixed files and data are checked against the supplied release hashes.

Install Python 3.12 and `requirements.txt`, obtain the exact repository version and matching checkpoint, then run from the repository root:

```
python verify_package.py
python -m unittest discover -s tests -v
python evaluate.py --checkpoint artifacts/final.pt \
  --device cpu --precision fp32 --threads 4 --split test
```

No retraining is required. The `assets/ngram.npz` file is part of the code bundle and required by the final predictor. The README gives full training, cache selection, statistical fitting, gating, freezing and resource-measurement commands. The source and checkpoint hashes recorded in `artifacts/FREEZE.json` were written before testing. Scores are verified using the same frozen files.

Final checkpoint SHA-256:
132eb36bf132406709b4610473b39ff9ae7400fb40520bffcc922f8dfb495cb2

Freeze timestamp (UTC): 2026-09-29T05:34:31.001780+00:00

6. Critical analysis, reuse and AI disclosure

Why combine the components? GPT generalizes across varied contexts. Exact short-sequence statistics provide a complementary estimate for recurring token patterns in the training corpus. A continuous cache adapts within the current document fragment without retaining information between windows. Their probabilities are mixed, preserving finite normalized outputs and the fixed scorer interface.

Costs and trade-offs. The statistical component uses substantially more inference storage than the small neural checkpoint and adds table-lookup and distribution-combination work. The cache adds an $O(B L^2 d)$ similarity computation, bounded by $L=256$, with sparse token aggregation. The largest neural candidate increases compute but remains small enough to be evaluated on CPU. The resource audit evaluates all three limits for the same frozen predictor.

Limits of the evidence. A single random seed is used, so the report does not establish robustness across seeds. Searching validation settings can overfit that split; the test score is a held-out assessment after freezing, not a tuning signal. Width, dropout, training device and schedule were not all independently controlled, so their individual causal effects cannot be separated. Optional per-order gating adds four hyperparameters and its ablation is needed to assess whether the extra selection is worthwhile. Timing and floating-point results may vary on other hardware. Improvements on this corpus and tokenizer do not establish general superiority.

Reused work. The DASE7506 starter supplies the GPT backbone, training recipe, benchmark, tokenizer, fixed scorer and original tests. `model.py`, `train.py`, `common.py`, `evaluate.py`, the baseline configuration and benchmark are retained unchanged. Additional trainers adapt the same recipe for checkpoint selection, continuation and MPS. The cache and smoothing concepts are prior work; the constrained implementation and experiment orchestration were written for this submission.

Substantive AI assistance. OpenAI Codex assisted with planning, implementing the model and experiment scripts, executing local training, testing, selecting settings on validation, analyzing results, drafting the report and reproduction instructions, and preparing submission materials. The student remains responsible for reviewing, understanding and explaining the work. This statement does not assert that independent student review has already occurred.

References and data attribution

- [1] E. Grave, A. Joulin, N. Usunier. Improving Neural Language Models with a Continuous Cache. ICLR 2017. <https://arxiv.org/abs/1612.04426>
- [2] R. Kneser, H. Ney. Improved Backing-Off for M-Gram Language Modeling. ICASSP 1995, pp. 181-184. <https://www-i6.informatik.rwth-aachen.de/publications/download/951/Kneser-ICASSP-1995.pdf>
- [3] S. F. Chen, J. Goodman. An Empirical Study of Smoothing Techniques for Language Modeling. Computer Speech and Language 13(4), 1999, pp. 359-394.
- [4] S. Merity, C. Xiong, J. Bradbury, R. Socher. Pointer Sentinel Mixture Models. ICLR 2017. <https://arxiv.org/abs/1609.07843>
- [5] DASE7506 MP1 supplied GUIDE.md, README.md and protocol 7506-mp1-wt2-v2.

WikiText text is by Wikipedia contributors. Retain the supplied CC BY-SA 3.0 and GNU Free Documentation License notices and dataset revision attribution when redistributing it. These data notices do not relicense the surrounding classroom source.